COMPLETE LEARNING ROADMAP

Complete Python Syllabus — Zero to Advanced + AIML Engineer

Complete Master Roadmap
Beginner to Advanced
Data Science + AI/ML

This syllabus covers Python from absolute basics to professional-level programming, coding interviews, Data Structures and Algorithms, Data Science, Machine Learning, Deep Learning and deployment.

It is structured chapter by chapter so you can use it as your complete Python study checklist.

7 STAGES	37 CHAPTERS	8 MONTHS
A progressive learning path	Core topics, practice and projects	Suggested study timeline

THE LEARNING PATH

STAGE	FOCUS	COVERAGE
STAGE 01	**Python Fundamentals** Beginner	Chapters 1-6
STAGE 02	**Python Data Structures** Beginner to Intermediate	Chapters 7-11
STAGE 03	**Functions and Core Python Programming** Intermediate	Chapters 12-20
STAGE 04	**Professional Python and Software Development** Intermediate to Advanced	Chapters 21-27
STAGE 05	**Python for Data Science** Data Science specialization	Chapters 28-31
STAGE 06	**Python for Machine Learning and Artificial Intelligence** AIML Engineer specialization	Chapters 32-35
STAGE 07	**Python Coding Mastery and Projects**	Chapters 36-37

Stages 1-6 cover the 35 core chapters; Stage 7 adds coding practice and project work (Chapters 36-37).

STAGE 01 / BEGINNER

Python Fundamentals

CHAPTER 01

Introduction to Python

- [ ] What is Python?
- [ ] History and features of Python
- [ ] Why Python is popular
- [ ] Applications of Python
- [ ] Python vs Java vs C++
- [ ] Compiled vs interpreted languages
- [ ] How Python works internally
- [ ] Python installation and setup
- [ ] Python interpreter and interactive shell
- [ ] IDE: VS Code, PyCharm, Jupyter Notebook
- [ ] First Python program
- [ ] Python execution process
- [ ] Source code, bytecode and PVM
- [ ] Comments and docstrings
- [ ] Python indentation and syntax
- [ ] print() and input()

CHAPTER 02

Variables, Constants and Memory

- [ ] What is a variable?
- [ ] Variable declaration and assignment
- [ ] Variable naming rules
- [ ] Identifiers and keywords
- [ ] Dynamic typing
- [ ] Static vs dynamic typing
- [ ] Multiple variable assignments
- [ ] Multiple-value unpacking
- [ ] Constants and naming conventions
- [ ] Object references
- [ ] Object identity and id()
- [ ] type() function
- [ ] is vs ==
- [ ] Memory management basics
- [ ] Variable scope introduction

CHAPTER 03

Python Data Types

- [ ] What are data types?
- [ ] Built-in data types
- [ ] Numeric types: int, float, complex
- [ ] Boolean: True, False
- [ ] String: str
- [ ] Sequence types
- [ ] List
- [ ] Tuple
- [ ] Range
- [ ] Set and frozenset

- [ ] Dictionary
- [ ] None type
- [ ] Mutable vs immutable
- [ ] Ordered vs unordered
- [ ] Hashable vs unhashable
- [ ] Type checking
- [ ] Type conversion
- [ ] Implicit vs explicit conversion
- [ ] Type errors and debugging

CHAPTER 04

Python Operators

- [ ] Arithmetic operators
- [ ] Comparison operators
- [ ] Assignment operators
- [ ] Logical operators
- [ ] Bitwise operators
- [ ] Membership operators
- [ ] Identity operators
- [ ] Operator precedence
- [ ] Operator associativity
- [ ] Short-circuit evaluation
- [ ] Chained comparisons

CHAPTER 05

Conditional Statements

- [ ] What is decision-making?
- [ ] if statement
- [ ] if-else
- [ ] if-elif-else
- [ ] Nested if statements
- [ ] Multiple conditions
- [ ] Logical operators with conditions
- [ ] Ternary conditional expression
- [ ] Truthy and falsy values
- [ ] match-case
- [ ] Nested conditional problems

CHAPTER 06

Loops and Iteration

- [ ] What are loops?
- [ ] for loop
- [ ] while loop
- [ ] range() function
- [ ] break statement
- [ ] continue statement
- [ ] pass statement
- [ ] Nested loops
- [ ] Loop with else
- [ ] Infinite loops
- [ ] Loop control
- [ ] Iteration over strings
- [ ] Iteration over lists, tuples, sets and dictionaries

- [ ] enumerate()
- [ ] zip()
- [ ] reversed()

STAGE 02 / BEGINNER TO INTERMEDIATE

Python Data Structures

CHAPTER 07

Python Strings

- [ ] What is a string?
- [ ] Creating strings
- [ ] Single, double and triple quotes
- [ ] String indexing
- [ ] Positive indexing
- [ ] Negative indexing
- [ ] String slicing
- [ ] String immutability
- [ ] String concatenation
- [ ] String repetition
- [ ] Membership operators
- [ ] String comparison
- [ ] String methods: upper(), lower(), title()
- [ ] strip(), lstrip(), rstrip()
- [ ] split() and join()
- [ ] replace()
- [ ] find(), index(), count()
- [ ] startswith() and endswith()
- [ ] isalpha(), isdigit(), isalnum()
- [ ] String formatting
- [ ] f-strings
- [ ] Escape characters
- [ ] Raw strings
- [ ] Unicode and encoding basics

CHAPTER 08

Python Lists

- [ ] What is a list?
- [ ] Creating lists
- [ ] List indexing
- [ ] List slicing
- [ ] Updating elements
- [ ] Adding elements
- [ ] append()
- [ ] extend()
- [ ] insert()
- [ ] Removing elements
- [ ] remove()
- [ ] pop()
- [ ] clear()
- [ ] del
- [ ] Searching and counting
- [ ] index() and count()
- [ ] Sorting and reversing
- [ ] sort() vs sorted()
- [ ] reverse() vs reversed()
- [ ] List concatenation and repetition

- [ ] Nested lists
- [ ] 2D and 3D lists
- [ ] List copying
- [ ] Shallow copy vs deep copy
- [ ] List aliasing
- [ ] List comprehension
- [ ] List methods time complexity

CHAPTER 09

Python Tuples

- [ ] What is a tuple?
- [ ] Creating tuples
- [ ] Empty and single-element tuples
- [ ] Tuple indexing and slicing
- [ ] Tuple immutability
- [ ] Tuple packing
- [ ] Tuple unpacking
- [ ] Extended unpacking
- [ ] Nested tuples
- [ ] Tuple methods
- [ ] count() and index()
- [ ] Tuple vs list
- [ ] Tuple hashability
- [ ] Tuple as dictionary key
- [ ] Named tuples
- [ ] Practical tuple applications

CHAPTER 10

Python Sets

- [ ] What is a set?
- [ ] Creating sets
- [ ] Empty set vs empty dictionary
- [ ] Unique elements
- [ ] Set mutability
- [ ] Adding elements
- [ ] add() and update()
- [ ] Removing elements
- [ ] remove(), discard(), pop(), clear()
- [ ] Union
- [ ] Intersection
- [ ] Difference
- [ ] Symmetric difference
- [ ] Subset and superset
- [ ] Disjoint sets
- [ ] Set comprehensions
- [ ] Frozenset
- [ ] Hashing and membership performance
- [ ] Practical set applications

CHAPTER 11

Python Dictionaries

- [ ] What is a dictionary?
- [ ] Key-value pairs

- [ ] Creating dictionaries
- [ ] Accessing values
- [ ] Adding and updating entries
- [ ] Deleting entries
- [ ] get()
- [ ] setdefault()
- [ ] update()
- [ ] pop() and popitem()
- [ ] keys(), values(), items()
- [ ] Dictionary iteration
- [ ] Nested dictionaries
- [ ] Dictionary comprehension
- [ ] Dictionary unpacking
- [ ] Dictionary merging
- [ ] Hashable dictionary keys
- [ ] defaultdict
- [ ] Counter
- [ ] OrderedDict
- [ ] Dictionary shallow and deep copy
- [ ] Dictionary time complexity

STAGE 03 / INTERMEDIATE

Functions and Core Python Programming

CHAPTER 12

Python Functions

- [ ] What is a function?
- [ ] Why functions are important
- [ ] Built-in vs user-defined functions
- [ ] Defining and calling functions
- [ ] Function syntax
- [ ] Parameters vs arguments
- [ ] Positional arguments
- [ ] Keyword arguments
- [ ] Default arguments
- [ ] Required arguments
- [ ] Return statement
- [ ] Multiple return values
- [ ] Functions returning None
- [ ] Variable-length arguments
- [ ] *args
- [ ] **kwargs
- [ ] Positional-only parameters
- [ ] Keyword-only parameters
- [ ] Function annotations
- [ ] Docstrings
- [ ] Local and global variables
- [ ] global keyword
- [ ] nonlocal keyword
- [ ] LEGB scope rule
- [ ] Lambda functions
- [ ] Recursion
- [ ] Higher-order functions
- [ ] First-class functions
- [ ] Pure functions
- [ ] Function composition

CHAPTER 13

Comprehensions and Functional Programming

- [ ] List comprehensions
- [ ] Conditional list comprehensions
- [ ] Nested comprehensions
- [ ] Dictionary comprehensions
- [ ] Set comprehensions
- [ ] Generator expressions
- [ ] map()
- [ ] filter()
- [ ] reduce()
- [ ] zip()
- [ ] enumerate()
- [ ] any() and all()
- [ ] sorted() with key
- [ ] min() and max() with key

- [ ] Functional programming concepts

CHAPTER 14
Modules and Packages

- [ ] What is a module?
- [ ] Creating custom modules
- [ ] Importing modules
- [ ] import
- [ ] from ... import
- [ ] Aliases with as
- [ ] Standard library modules
- [ ] math
- [ ] random
- [ ] datetime
- [ ] os
- [ ] sys
- [ ] collections
- [ ] itertools
- [ ] functools
- [ ] pathlib
- [ ] Creating packages
- [ ] __init__.py
- [ ] Module search path
- [ ] __name__
- [ ] if __name__ == "__main__"

CHAPTER 15
Exception Handling

- [ ] What is an exception?
- [ ] Syntax errors vs runtime errors
- [ ] Common Python exceptions
- [ ] try
- [ ] except
- [ ] Multiple except blocks
- [ ] else
- [ ] finally
- [ ] raise
- [ ] Custom exceptions
- [ ] Exception hierarchy
- [ ] Exception chaining
- [ ] Assertions
- [ ] Debugging techniques

CHAPTER 16
File Handling

- [ ] What is file handling?
- [ ] Opening files
- [ ] open()
- [ ] Read mode
- [ ] Write mode
- [ ] Append mode
- [ ] read()
- [ ] readline()

- [ ] readlines()
- [ ] write()
- [ ] writelines()
- [ ] seek() and tell()
- [ ] close()
- [ ] Context manager (with)
- [ ] Text vs binary files
- [ ] CSV file handling
- [ ] JSON file handling
- [ ] File and directory paths
- [ ] pathlib
- [ ] File exceptions

CHAPTER 17

Object-Oriented Programming (OOP)

- [ ] What is OOP?
- [ ] Class and object
- [ ] Attributes and methods
- [ ] __init__()
- [ ] self
- [ ] Instance variables
- [ ] Class variables
- [ ] Instance methods
- [ ] Class methods
- [ ] Static methods
- [ ] Encapsulation
- [ ] Public, protected and private conventions
- [ ] Inheritance
- [ ] Single inheritance
- [ ] Multiple inheritance
- [ ] Multilevel inheritance
- [ ] Hierarchical inheritance
- [ ] Hybrid inheritance
- [ ] super()
- [ ] Method overriding
- [ ] Polymorphism
- [ ] Duck typing
- [ ] Operator overloading
- [ ] Abstraction
- [ ] Abstract classes
- [ ] Properties and @property
- [ ] Getters and setters
- [ ] MRO (Method Resolution Order)
- [ ] Magic/dunder methods
- [ ] __str__, __repr__, __len__
- [ ] Dataclasses
- [ ] Composition vs inheritance

CHAPTER 18

Advanced Python Programming

- [ ] Iterable vs iterator
- [ ] iter() and next()

- [ ] Creating custom iterators
- [ ] Generators
- [ ] yield vs return
- [ ] Generator expressions
- [ ] Generator pipelines
- [ ] Memory-efficient iteration
- [ ] Decorators
- [ ] Decorators with arguments
- [ ] Multiple decorators
- [ ] Closures
- [ ] Nested functions
- [ ] First-class functions
- [ ] Shallow copy
- [ ] Deep copy
- [ ] copy module
- [ ] Context managers
- [ ] Creating custom context managers
- [ ] with statement
- [ ] property and descriptors
- [ ] __slots__
- [ ] Metaclasses introduction
- [ ] Dynamic attribute access
- [ ] Reflection and introspection

CHAPTER 19

Python Memory Management and Internals

- [ ] How Python manages memory
- [ ] Variables as object references
- [ ] Namespaces
- [ ] Object identity
- [ ] Reference counting
- [ ] Garbage collection
- [ ] Cyclic references
- [ ] Memory allocation
- [ ] Stack and heap concepts
- [ ] Mutable vs immutable memory behavior
- [ ] Object interning
- [ ] Integer and string caching
- [ ] LEGB scope resolution
- [ ] Global and local namespaces
- [ ] Python execution model
- [ ] CPython vs PyPy
- [ ] GIL (Global Interpreter Lock)
- [ ] Memory optimization techniques
- [ ] Profiling memory usage

CHAPTER 20

Python Regular Expressions

- [ ] What is Regex?
- [ ] re module
- [ ] Literal characters
- [ ] Metacharacters

- [ ] Character classes
- [ ] Quantifiers
- [ ] Anchors
- [ ] Groups and capturing
- [ ] Non-capturing groups
- [ ] Lookahead and lookbehind
- [ ] re.match()
- [ ] re.search()
- [ ] re.findall()
- [ ] re.finditer()
- [ ] re.sub()
- [ ] re.split()
- [ ] Email and phone validation
- [ ] Text cleaning using regex
- [ ] Regex for dataset preprocessing

STAGE 04 / INTERMEDIATE TO ADVANCED

Professional Python and Software Development

CHAPTER 21

Python Debugging and Testing

- [ ] Understanding error messages
- [ ] Debugging techniques
- [ ] Using breakpoints
- [ ] Python debugger (pdb)
- [ ] Logging module
- [ ] Logging levels
- [ ] Assertions
- [ ] Unit testing
- [ ] unittest
- [ ] pytest
- [ ] Test cases
- [ ] Fixtures
- [ ] Parameterized testing
- [ ] Mocking
- [ ] Code coverage
- [ ] Debugging real-world projects

CHAPTER 22

Python Virtual Environments and Dependency Management

- [ ] Python environment
- [ ] Why virtual environments are needed
- [ ] venv
- [ ] Creating environments
- [ ] Activating and deactivating environments
- [ ] Installing libraries using pip
- [ ] Uninstalling packages
- [ ] Updating packages
- [ ] requirements.txt
- [ ] Dependency version management
- [ ] pip freeze
- [ ] pyproject.toml
- [ ] Package management with Poetry basics
- [ ] Environment variables
- [ ] .env files
- [ ] Reproducible environments

CHAPTER 23

Python Type Hints and Code Quality

- [ ] Static vs dynamic typing
- [ ] Type annotations
- [ ] Function type hints
- [ ] Return type hints
- [ ] list[int], dict[str, int]
- [ ] Optional and union types
- [ ] Any and None
- [ ] TypedDict
- [ ] Protocols

- [ ] Generic types
- [ ] Type checking with mypy basics
- [ ] PEP 8 coding standards
- [ ] Clean code principles
- [ ] Naming conventions
- [ ] Code refactoring
- [ ] Documentation and docstrings
- [ ] Code formatting with Black
- [ ] Linting with Ruff

CHAPTER 24

Python Concurrency and Parallelism

- [ ] What is concurrency?
- [ ] What is parallelism?
- [ ] Synchronous vs asynchronous programming
- [ ] Processes vs threads
- [ ] Multithreading
- [ ] threading module
- [ ] Thread synchronization
- [ ] Locks and deadlocks
- [ ] Multiprocessing
- [ ] multiprocessing module
- [ ] Process pools
- [ ] concurrent.futures
- [ ] ThreadPoolExecutor
- [ ] ProcessPoolExecutor
- [ ] GIL and concurrency
- [ ] Async programming
- [ ] async and await
- [ ] asyncio
- [ ] Async tasks
- [ ] Async HTTP requests
- [ ] When to use threads vs processes vs async

CHAPTER 25

Networking, APIs and Database Programming

- [ ] HTTP fundamentals
- [ ] HTTP methods: GET, POST, PUT, PATCH, DELETE
- [ ] HTTP status codes
- [ ] JSON request and response
- [ ] REST APIs
- [ ] Requests library
- [ ] Sending API requests
- [ ] API authentication
- [ ] API keys and tokens
- [ ] Handling API errors
- [ ] Pagination
- [ ] Rate limiting
- [ ] Working with SQL databases
- [ ] SQLite
- [ ] MySQL/PostgreSQL basics
- [ ] SQL queries using Python

- [ ] Database connections
- [ ] CRUD operations
- [ ] SQLAlchemy basics
- [ ] Transactions
- [ ] MongoDB and PyMongo basics
- [ ] Database integration in AIML applications

CHAPTER 26

Data Structures and Algorithms (DSA) Using Python

- [ ] What is DSA?
- [ ] Time complexity
- [ ] Space complexity
- [ ] Big O notation
- [ ] Best, average and worst cases
- [ ] Arrays and dynamic arrays
- [ ] Strings
- [ ] Hash maps and hash sets
- [ ] Two pointers
- [ ] Sliding window
- [ ] Prefix sums
- [ ] Searching algorithms
- [ ] Linear search
- [ ] Binary search
- [ ] Sorting algorithms
- [ ] Bubble sort
- [ ] Selection sort
- [ ] Insertion sort
- [ ] Merge sort
- [ ] Quick sort
- [ ] Recursion
- [ ] Backtracking
- [ ] Stack
- [ ] Queue and deque
- [ ] Linked lists
- [ ] Trees and binary trees
- [ ] Binary search trees
- [ ] Heaps and priority queues
- [ ] Hashing
- [ ] Graphs
- [ ] BFS and DFS
- [ ] Greedy algorithms
- [ ] Dynamic programming
- [ ] Memoization and tabulation

CHAPTER 27

Advanced Python Standard Libraries

- [ ] collections
- [ ] itertools
- [ ] functools
- [ ] operator
- [ ] heapq
- [ ] bisect

- [ ] statistics
- [ ] decimal
- [ ] fractions
- [ ] math
- [ ] random
- [ ] secrets
- [ ] datetime
- [ ] time
- [ ] calendar
- [ ] json
- [ ] csv
- [ ] pickle
- [ ] sqlite3
- [ ] logging
- [ ] argparse
- [ ] subprocess
- [ ] shutil
- [ ] glob
- [ ] pathlib
- [ ] re
- [ ] uuid

STAGE 05 / DATA SCIENCE SPECIALIZATION

Python for Data Science

CHAPTER 28

NumPy — Numerical Computing

- [ ] What is NumPy?
- [ ] NumPy installation
- [ ] NumPy vs Python lists
- [ ] Creating arrays
- [ ] array(), arange(), linspace()
- [ ] Zeros, ones and identity matrices
- [ ] Array dimensions
- [ ] ndim, shape, size, dtype
- [ ] Reshape and flatten
- [ ] Indexing and slicing
- [ ] Boolean indexing
- [ ] Fancy indexing
- [ ] Array operations
- [ ] Broadcasting
- [ ] Vectorization
- [ ] Universal functions
- [ ] Aggregation functions
- [ ] Axis 0 vs axis 1
- [ ] Sorting and searching
- [ ] Joining and splitting arrays
- [ ] Copy vs view
- [ ] Random number generation
- [ ] Missing values (NaN)
- [ ] Linear algebra
- [ ] Dot product
- [ ] Matrix multiplication
- [ ] Transpose and inverse
- [ ] Eigenvalues and eigenvectors
- [ ] NumPy for ML feature matrices

CHAPTER 29

Pandas — Data Analysis

- [ ] What is Pandas?
- [ ] Series and DataFrame
- [ ] Creating DataFrames
- [ ] Reading CSV and Excel files
- [ ] head(), tail(), sample()
- [ ] info() and describe()
- [ ] Selecting columns
- [ ] Selecting rows
- [ ] loc vs iloc
- [ ] Filtering and conditional selection
- [ ] Sorting data
- [ ] Adding and deleting columns
- [ ] Renaming columns
- [ ] Handling missing values
- [ ] isnull() and notnull()

- [ ] fillna() and dropna()
- [ ] Removing duplicates
- [ ] Data type conversion
- [ ] astype()
- [ ] apply() and map()
- [ ] groupby()
- [ ] Aggregation
- [ ] value_counts()
- [ ] merge()
- [ ] join()
- [ ] concat()
- [ ] Pivot tables
- [ ] Cross-tabulation
- [ ] String operations
- [ ] Date and time operations
- [ ] CSV and Excel export
- [ ] Efficient handling of large datasets
- [ ] End-to-end data cleaning

CHAPTER 30

Data Visualization

- [ ] Data visualization fundamentals
- [ ] Matplotlib introduction
- [ ] Figure and axes
- [ ] Line charts
- [ ] Bar charts
- [ ] Horizontal bar charts
- [ ] Scatter plots
- [ ] Histograms
- [ ] Pie charts
- [ ] Boxplots
- [ ] Subplots
- [ ] Labels and legends
- [ ] Titles and annotations
- [ ] Saving figures
- [ ] Seaborn introduction
- [ ] Distribution plots
- [ ] Count plots
- [ ] Boxplots and violin plots
- [ ] Pair plots
- [ ] Heatmaps
- [ ] Correlation matrices
- [ ] Outlier visualization
- [ ] Data storytelling
- [ ] Choosing the right chart

CHAPTER 31

Statistics and Mathematics for AIML

- [ ] Mean, median and mode
- [ ] Range and variance
- [ ] Standard deviation
- [ ] Percentiles and quartiles

- [ ] IQR
- [ ] Outlier detection
- [ ] Probability fundamentals
- [ ] Conditional probability
- [ ] Bayes' theorem
- [ ] Probability distributions
- [ ] Normal distribution
- [ ] Binomial distribution
- [ ] Poisson distribution
- [ ] Uniform distribution
- [ ] Sampling
- [ ] Central Limit Theorem
- [ ] Hypothesis testing
- [ ] Null and alternative hypotheses
- [ ] P-value
- [ ] Confidence intervals
- [ ] Correlation and covariance
- [ ] Linear algebra
- [ ] Vectors and matrices
- [ ] Matrix operations
- [ ] Calculus fundamentals
- [ ] Derivatives and gradients
- [ ] Gradient descent
- [ ] Optimization basics

STAGE 06 / AIML ENGINEER SPECIALIZATION

Python for Machine Learning and Artificial Intelligence

CHAPTER 32

Data Preprocessing and Feature Engineering

- [ ] Complete ML data preprocessing workflow
- [ ] Data collection
- [ ] Data inspection
- [ ] Data quality checks
- [ ] Missing value treatment
- [ ] Duplicate removal
- [ ] Outlier detection and treatment
- [ ] Categorical vs numerical features
- [ ] Label encoding
- [ ] One-hot encoding
- [ ] Ordinal encoding
- [ ] Feature scaling
- [ ] Standardization
- [ ] Normalization
- [ ] Robust scaling
- [ ] Train, validation and test split
- [ ] Stratified sampling
- [ ] Data leakage
- [ ] Feature selection
- [ ] Feature extraction
- [ ] Feature transformation
- [ ] Polynomial features
- [ ] Handling imbalanced datasets
- [ ] SMOTE fundamentals
- [ ] Scikit-learn preprocessing pipelines
- [ ] ColumnTransformer

CHAPTER 33

Machine Learning with Python

- [ ] What is Machine Learning?
- [ ] Types of Machine Learning
- [ ] ML lifecycle
- [ ] Scikit-learn introduction
- [ ] Dataset loading
- [ ] Features and target variables
- [ ] X and y
- [ ] Train/test split
- [ ] Model fitting
- [ ] Prediction
- [ ] Model evaluation

Supervised learning

- [ ] Linear regression
- [ ] Multiple linear regression
- [ ] Polynomial regression
- [ ] Ridge regression
- [ ] Lasso regression

- [ ] Logistic regression
- [ ] K-Nearest Neighbors
- [ ] Decision trees
- [ ] Random forests
- [ ] Support Vector Machines
- [ ] Naive Bayes
- [ ] Gradient boosting
- [ ] XGBoost
- [ ] LightGBM
- [ ] CatBoost

Unsupervised learning

- [ ] K-Means clustering
- [ ] Hierarchical clustering
- [ ] DBSCAN
- [ ] PCA
- [ ] Dimensionality reduction
- [ ] Anomaly detection

Model evaluation

- [ ] Confusion matrix
- [ ] Accuracy
- [ ] Precision
- [ ] Recall
- [ ] F1-score
- [ ] Specificity
- [ ] ROC curve
- [ ] AUC
- [ ] MAE
- [ ] MSE
- [ ] RMSE
- [ ] R-squared
- [ ] Cross-validation
- [ ] Hyperparameter tuning
- [ ] GridSearchCV
- [ ] RandomizedSearchCV
- [ ] Overfitting and underfitting
- [ ] Bias-variance tradeoff
- [ ] Model selection
- [ ] Model persistence using Joblib and Pickle
- [ ] End-to-end ML pipeline

CHAPTER 34

Deep Learning with Python

- [ ] What is Deep Learning?
- [ ] Machine Learning vs Deep Learning
- [ ] Artificial neural networks
- [ ] Neurons and layers
- [ ] Input, hidden and output layers
- [ ] Weights and biases
- [ ] Forward propagation
- [ ] Loss functions
- [ ] Backpropagation

- [ ] Gradient descent
- [ ] Learning rate
- [ ] Optimizers
- [ ] SGD
- [ ] Adam
- [ ] Activation functions
- [ ] ReLU
- [ ] Sigmoid
- [ ] Tanh
- [ ] Softmax
- [ ] Binary and categorical cross-entropy
- [ ] Epochs, batch size and iterations
- [ ] Weight initialization
- [ ] Dropout
- [ ] Batch normalization
- [ ] Regularization
- [ ] Early stopping
- [ ] GPU and CPU training
- [ ] TensorFlow and Keras
- [ ] PyTorch
- [ ] Tensors and tensor operations
- [ ] Automatic differentiation
- [ ] Training loops
- [ ] Saving and loading neural networks

Advanced neural networks

- [ ] Artificial Neural Networks (ANN)
- [ ] Convolutional Neural Networks (CNN)
- [ ] Recurrent Neural Networks (RNN)
- [ ] Long Short-Term Memory (LSTM)
- [ ] Gated Recurrent Units (GRU)
- [ ] Autoencoders
- [ ] Generative Adversarial Networks (GANs)
- [ ] Attention mechanisms
- [ ] Transformers
- [ ] Transfer learning
- [ ] Fine-tuning
- [ ] Model optimization

CHAPTER 35

NLP, Generative AI, Deployment and MLOps

Natural Language Processing

- [ ] What is NLP?
- [ ] Text preprocessing
- [ ] Tokenization
- [ ] Stopword removal
- [ ] Stemming
- [ ] Lemmatization
- [ ] Bag of Words
- [ ] TF-IDF
- [ ] Word embeddings
- [ ] Word2Vec
- [ ] GloVe

- [ ] Sentiment analysis
- [ ] Text classification
- [ ] Named Entity Recognition
- [ ] Hugging Face Transformers
- [ ] BERT and related models

Generative AI and LLMs

- [ ] What is Generative AI?
- [ ] Large Language Models
- [ ] Transformer architecture
- [ ] Tokens and tokenization
- [ ] Embeddings
- [ ] Prompt engineering
- [ ] Context windows
- [ ] Temperature and sampling
- [ ] LLM APIs
- [ ] Retrieval-Augmented Generation (RAG)
- [ ] Vector databases
- [ ] Semantic search
- [ ] Chunking and retrieval
- [ ] Reranking
- [ ] Fine-tuning fundamentals
- [ ] AI agents and tool calling
- [ ] LangChain and LangGraph fundamentals
- [ ] LLM evaluation and hallucination checks
- [ ] Responsible AI and safety

Model deployment

- [ ] Model serialization
- [ ] REST API development
- [ ] Flask
- [ ] FastAPI
- [ ] Streamlit
- [ ] Model serving
- [ ] Input validation
- [ ] API testing
- [ ] Docker fundamentals
- [ ] Cloud deployment basics

MLOps

- [ ] ML lifecycle management
- [ ] Experiment tracking
- [ ] MLflow
- [ ] Model versioning
- [ ] Data versioning
- [ ] Model monitoring
- [ ] Data drift
- [ ] Model drift
- [ ] Retraining
- [ ] CI/CD basics
- [ ] Model performance monitoring
- [ ] Production ML pipelines

STAGE 07

Python Coding Mastery and Projects

These are not optional if your goal is to become confident in programming and clear AIML coding interviews.

CHAPTER 36

Python problem-solving question bank

Topic	Recommended practice questions
Variables and operators	20
Conditional statements	20
Loops and patterns	30
Strings	40
Lists and arrays	50
Tuples	15
Sets	20
Dictionaries	40
Functions	40
Recursion	20
Lambda and comprehensions	20
OOP	30
Exception handling	15
File handling	15
Iterators and generators	20
Decorators and closures	15
DSA	100
NumPy and Pandas	50
AIML coding problems	50

> These are recommended practice targets, not a claim that all questions are supplied in this syllabus.

CHAPTER 37

Practical Python projects

Beginner projects

- Calculator
- Number guessing game
- Student management system
- Expense tracker
- Contact management system

Intermediate projects

- Sales data analysis
- Employee salary analysis

- Student performance dashboard
- Weather data analysis
- Exploratory data analysis project

Machine Learning projects

- House price prediction
- Loan approval prediction
- Customer churn prediction
- Customer segmentation
- Fake news detection

Advanced AIML projects

- Crop disease detection
- Face recognition
- Recommendation system
- Image classification
- NLP chatbot
- RAG-based document assistant

8. Recommended study order and timeline

For your AIML engineering goal, follow this priority order:

Priority	Topics	Importance
1	Chapters 1–11: Python fundamentals and data structures	Essential
2	Chapters 12–19: Functions, OOP and advanced Python	Essential
3	Chapters 20–27: Professional Python and DSA	Very important
4	Chapters 28–31: NumPy, Pandas and Statistics	Essential for AIML
5	Chapters 32–33: Preprocessing and Machine Learning	Essential
6	Chapters 34–35: Deep Learning and Generative AI	Advanced specialization
7	Chapters 36–37: Coding practice and projects	Essential for mastery

Suggested timeline

Period	Target
Month 1	Python fundamentals
Month 2	Data structures and functions
Month 3	OOP, advanced Python and DSA
Month 4	NumPy, Pandas and visualization
Month 5	Statistics and Machine Learning
Month 6	Deep Learning and projects
Month 7–8	Generative AI, deployment and interview preparation

This is an 8-month suggested plan at around 2–3 hours daily. Adjust it according to your existing knowledge and practice speed.

9. Rules for mastering every chapter

For every chapter, use this exact learning structure:

01. Complete theory and definitions.

02. Understand syntax and every keyword.
03. Learn beginner-level programs.
04. Learn intermediate and advanced programs.
05. Perform line-by-line dry runs.
06. Understand memory and internal execution.
07. Solve practical real-world problems.

8. Solve 20–40 coding questions, depending on the topic.

9. Learn time and space complexity.

10. Practice interview questions and answers.

11. Complete a chapter test.
12. Build a small project using the concept.

> One important distinction: Understanding code is not the same as mastering code. Mastery means you can solve a new problem without looking at the solution, explain your approach, debug errors and optimize your program.

10. Your master syllabus completion tracker

Use the 7 stages as your overall progress targets:

Course progress

0% COMPLETE

☐ **Stage 1: Fundamentals**	Chapters 1–6
☐ **Stage 2: Data Structures**	Chapters 7–11
☐ **Stage 3: Core Python**	Chapters 12–19
☐ **Stage 4: Professional Python**	Chapters 20–27
☐ **Stage 5: Data Science**	Chapters 28–31
☐ **Stage 6: AIML**	Chapters 32–35
☐ **Stage 7: Coding and Projects**	Chapters 36–37

My recommendation: Start with Chapter 1 and do not skip chapters. Build a strong foundation in Python data structures, functions, OOP and problem-solving before moving to Machine Learning.

The syllabus above is your complete 37-chapter learning roadmap. We can use it as the structure for your detailed lessons, including code, dry runs, practical applications, exercises and solutions.